Visitor Management System

ServiceNow Portfolio Project — Full Build Documentation (CSA / CAD Skills Demonstration)

A custom scoped ServiceNow application built to manage the full visitor lifecycle — from pre-registration through check-in, host approval, and check-out — designed to demonstrate ServiceNow System Administrator (CSA) and Certified Application Developer (CAD) skills through a real, working application rather than certifications alone. This document walks through every phase of the build, in the order it was completed, along with the technical decisions and debugging discoveries made along the way.

Prepared by	Nikhil
Platform	ServiceNow (Personal Developer Instance)
Certifications	CSA, CAD
Repository	https://github.com/nikhilkotha29/visitor-management-servicenow

Table of Contents

Phase 1 — Setup and Planning

Established the ServiceNow scoped application and sketched the data model before building anything.

Steps completed

- Requested a Personal Developer Instance (PDI) from developer.servicenow.com
- Created a new Scoped Application in Studio named “Visitor Management” with auto-generated scope prefix x_1958858_visito_0
- Verified the application scope context was active before beginning any table or logic work
- Sketched the initial data model: Visitor, Host (mapped to sys_user), and Visit, later expanded to include Invitation as a separate table
- Decided to include two enhancement features early: an approval workflow for VIP/Contractor visitor types, and QR-code invitations for faster kiosk check-in
- Decided to model Invitation as its own table (rather than extra fields on Visit) to support a clean one-to-many relationship between a host's invitations and eventual visits, including support for walk-ins with no invitation record

Phase 2 — Tables and Data Model

Built the three core tables that make up the application.

Visitor table (x_1958858_visito_0_visitor)

- First Name, Last Name, Email — mandatory String/Email fields
- Phone, Company — optional String fields
- Visitor Type — Choice field: Guest, VIP, Contractor, Vendor (default Guest)
- Photo — optional Image field
- ID Verified — True/False checkbox for front desk use

Invitation table (x_1958858_visito_0_invitation)

- Visitor — Reference to Visitor (mandatory)
- Host — Reference to sys_user (mandatory)
- Expected Visit Date/Time — mandatory Date/Time field
- QR Token — auto-generated, read-only, unique-indexed String field
- Status — Choice: Pending, Used, Expired, Cancelled
- Purpose of Visit — Choice: Meeting, Interview, Delivery, Other
- Notes — optional String field

Visit table (x_1958858_visito_0_visit)

- Visitor — Reference to Visitor (mandatory)
- Host — Reference to sys_user (mandatory)
- Invitation — Reference to Invitation (optional; empty indicates a walk-in)
- Purpose of Visit — Choice field
- Approval Status — Choice: Not Required, Pending, Approved, Rejected
- Check-in Time / Check-out Time — Date/Time fields, read-only on the form (system-set)
- Status — Choice: Checked In, Checked Out, Awaiting Approval, No-Show
- Badge Number — optional String field
- Location/Site — Reference to cmn_location

Relationship summary: Visitor has many Invitations and many Visits. A host (sys_user) has many Invitations and many Visits. An Invitation optionally converts into exactly one Visit.

Phase 3 — Security (Access Control Lists)

Implemented role-based and record-level scripted access control to enforce separation of duties between front desk staff, hosts, and admins.

Custom roles created

- front_desk — full visitor/visit management for reception staff
- host — restricted to records where the user is the host
- admin — full control including deletion and configuration

ACL design per table

Visitor table: role-based ACLs only (create/write restricted to front_desk and admin; read open to front_desk, host, and admin; delete restricted to admin).

Invitation and Visit tables: read and write ACLs use a scripted condition so a host can only see or act on their own records, while front_desk and admin retain broader access:

```
answer = (current.host == gs.getUserID())
  || gs.hasRole('front_desk')
  || gs.hasRole('admin');
```

For Visit specifically, the write ACL additionally restricts a host to editing only while the record's approval_status is Pending — once approved or rejected, the host can no longer modify it.

Testing process

Built a full test matrix covering test.host1, test.host2, test.frontdesk, an admin account, and a user with no application role, verifying create/read/write/delete behavior for each role against each table. Used Security Debug logging and direct-URL record access (not just list-view filtering) to confirm ACLs were genuinely enforced rather than just visually hidden.

Issues found and resolved during this phase

- A module (Invitations) was invisible to the host role because its Roles field referenced a different role name (invitation_user) than the one assigned to test users — resolved by aligning role names across modules, ACLs, and user assignments
- Host1 could see both host1's and host2's invitations despite a correctly scripted read ACL — traced to a second, auto-generated ACL (role-only, no script) that ServiceNow creates by default when a table is built. Since ServiceNow grants access if any applicable ACL passes, this second ACL silently bypassed the record-level restriction. Resolved by deactivating the auto-generated ACLs on Invitation, Visitor, and Visit tables across all four operations (create/read/write/delete)
- After fixing ACLs, front desk lost access entirely due to the front_desk role never actually being assigned to the test.frontdesk user account — assigning the role resolved this

Phase 4 — Business Logic (Business Rules and Script Includes)

Implemented the core business logic governing check-in behavior, approval requirements, and data integrity, centralizing reusable logic in a Script Include.

Script Include: VMSUtils

- generateQRToken() — generates a unique QR token for an Invitation
- isVisitorCurrentlyCheckedIn(visitorSysId) — checks whether a visitor already has an open (Checked In) visit, used to prevent duplicate check-ins
- requiresApproval(visitorTypeValue) — returns true for VIP or Contractor visitor types

Business Rules built (on the Visit table unless noted)

- Prevent Duplicate Check-ins — before insert, Order 50 (runs first): aborts the insert with an error message if the visitor already has an open Checked In visit
- Set Approval Status on Insert — before insert, Order 100: sets approval_status to Pending for VIP/Contractor visitor types, otherwise Not Required
- Auto-set Check-in Time — before insert, Order 200: if approval is not required, sets status to Checked In and populates check_in_time; otherwise sets status to Awaiting Approval
- Auto-set Check-out Time — before update, condition Status changes to Checked Out: includes a guard that blocks the checkout (via setAbortAction) unless the record's previous status was Checked In, then populates check_out_time
- Mark Invitation as Used on Check-in — after insert, when linked to an Invitation: updates the linked Invitation's status to Used
- On the Invitation table — before insert: auto-generates the QR Token via VMSUtils

Business Rule execution order

Explicit Order values were required because ServiceNow does not guarantee rule execution in creation order. The duplicate-check guard (Order 50) must run before the approval-status setter (Order 100), which must run before the check-in-time setter (Order 200) — otherwise the check-in logic would evaluate an empty approval_status and incorrectly auto-check-in VIP/Contractor visitors.

Testing performed

- Guest visit: confirmed approval_status = Not Required, status = Checked In, check-in time populated
- VIP visit: confirmed approval_status = Pending, status = Awaiting Approval, check-in time empty
- Checkout guard: confirmed a non-checked-in visit cannot be marked Checked Out, and a genuinely checked-in visit can be checked out normally
- Duplicate check-in: confirmed a second Visit for an already-checked-in visitor is blocked with an error and does not save

Phase 5 — Automation (Flow Designer and Scheduled Jobs)

Flow A — Host Approval Request

Trigger: Visit record created where Approval Status is Pending.

1. Ask For Approval action on the Visit record, with the Host field set as the approver for both the Approve and Reject rule sets
2. If branch (approval state is Approved): Update Record sets Approval Status = Approved, Status = Checked In, and populates Check-in Time
3. Else branch (rejected): Update Record sets Approval Status = Rejected and Status = No-Show

Tested end-to-end: a VIP visit approved by the host correctly updated to Approved / Checked In with a populated check-in time; a separate VIP visit rejected by the host correctly updated to Rejected / No-Show with check-in time remaining empty.

Flow B — QR Invitation Email

Trigger: Invitation record created. A Send Email action emails the visitor with visit details (host name, expected date/time, purpose) and the QR token, using data pills to merge real field values into the subject and body. Verified by creating a test invitation and confirming the generated email record contained correctly merged values.

Scheduled Job — Auto Checkout Stale Visits

Runs daily and auto-checks-out any Visit still in Checked In status more than 12 hours after its check-in time:

```
var gr = new GlideRecord('x_1958858_visito_0_visit');
gr.addQuery('status', 'Checked In');
gr.addQuery('check_in_time', '<', gs.hoursAgo(12));
gr.query();
while (gr.next()) {
  gr.setValue('status', 'Checked Out');
  gr.update();
}
```

Tested by backdating a Checked In visit's check-in time via a background script (with setWorkflow(false) to bypass Business Rules during the manual backdate) and running the job via Execute Now, confirming the record was correctly auto-checked-out.

Phase 6 — User Interface (Service Portal)

Built a front-desk-facing Service Portal page with two widgets.

VMS Check-in Form widget

An AngularJS-based widget with fields for First Name, Last Name, Email, Company, Visitor Type, a Host search field, and Purpose of Visit. The server script creates a Visitor record, looks up the host by a CONTAINS name match, and creates the linked Visit record, returning a success or error message back to the form.

VMS Currently Checked In widget

A read-only table widget querying all Visit records with Status = Checked In, displaying Visitor (built from first_name + last_name), Host, Check-in Time, and Purpose.

Build and testing notes

- Portal and page were created via Service Portal Designer at a custom URL suffix; verified the correct page (not the platform's default “sp” portal homepage) was being loaded
- Fixed a widget controller lint error by naming the AngularJS controller function rather than leaving it anonymous
- Fixed a runtime error caused by the widget referencing a Visitor field's display value when the Visitor table had no Display field configured, initially showing raw sys_ids instead of visitor names — resolved by concatenating first_name and last_name directly in the server script
- Verified end-to-end: submitted a check-in through the portal, confirmed the Visitor and Visit records were created correctly in the backend, and confirmed the Currently Checked In list reflected the new entry after a page refresh

Phase 7 — Automated Test Framework (ATF)

Built four regression tests, chosen for coverage of business logic and security rather than volume.

Test 1 — Guest visit auto check-in

Impersonates front desk, creates a Guest Visitor and a Visit, then asserts Approval Status is Not Required, Status is Checked In, and Check-in Time is populated. Uses an Open an Existing Record step before the Field Values Validation assertion step, since that assertion type requires an open form context.

Test 2 — VIP visit requires approval

Same structure as Test 1, but with a VIP Visitor Type, asserting Approval Status is Pending, Status is Awaiting Approval, and Check-in Time is empty. This test directly guards against a regression of the Business Rule ordering issue found in Phase 4.

Test 3 — Host cannot access other host's invitation

Creates an Invitation with Host = test.host2 while impersonating front desk, then switches impersonation to test.host1 and uses a Run Server Side Script step to check access. Initially used GlideRecord.get() to check access, which returned true even though it should have been blocked; switching to GlideRecord.canRead() correctly evaluated ACLs for the impersonated session and passed the assertion.

Test 4 — Duplicate check-in prevention

Creates a Guest Visitor and an initial Visit (Checked In), then attempts a second Visit insert for the same visitor via a Run Server Side Script step, asserting the insert is blocked. This test caught a real regression where the VMSUtils.isVisitorCurrentlyCheckedIn function still queried for the lowercase value checked_in rather than the actual choice value Checked In, silently disabling duplicate-check-in prevention.

Result

All four tests pass and were grouped for one-click execution as a demonstration of a working regression suite.

Phase 8 — Reporting and Dashboard

Built four reports and assembled them into a single dashboard.

- Visits by Purpose — pie chart grouped by Purpose of Visit
- Visits by Status — bar chart grouped by Status
- Average Visit Duration — single score report averaging a Duration (Minutes) field, calculated and stored by the check-out Business Rule using GlideDateTime subtraction between check-in and check-out times, filtered to Status = Checked Out
- Currently Checked In Count — single score report filtered to Status = Checked In

All four widgets were added to a Visitor Management Dashboard and verified to display real data reflecting the visits created during testing.

Phase 9 — Packaging and Showcase

Source control

ServiceNow Studio's Source Control integration was used to push the scoped application to a GitLab repository, capturing all tables, business rules, flows, ACLs, and widgets as XML.

GitHub showcase repository

A separate, curated GitHub repository was created to present the project publicly, including:

- This documentation PDF
- A README.md summarizing the project with embedded screenshots
- An exported Update Set XML as a portable artifact
- Screenshots of the Service Portal, the approval flow, ACL configuration, passing ATF tests, and the reporting dashboard
- A data model diagram showing the Visitor, Invitation, Visit, and sys_user relationships

Demo video

A short walkthrough video was planned covering: introduction and architecture, the core check-in workflow through the Service Portal, the VIP approval workflow, a brief note on the ACL security model, the ATF test suite passing, and the reporting dashboard.

Debugging Stories

Three issues were found and resolved during development that are worth highlighting, since working through them demonstrated real platform understanding beyond following a tutorial.

1. ACL OR-evaluation conflict

ServiceNow auto-generates a default, role-only ACL set whenever a table is created. These silently coexisted with custom scripted ACLs built during Phase 3. Since ServiceNow grants access if any matching ACL passes, the permissive auto-generated ACL let every host see every other host's records, bypassing the intended record-level restriction entirely. Fixed by auditing every ACL on each table and deactivating the auto-generated ones, leaving one authoritative scripted ACL per table and operation.

2. Choice value casing mismatch

Scripts across several Business Rules, a Script Include, and a Scheduled Job assumed snake_case choice values such as checked_in. The actual configured choice list used space-separated, capitalized values such as Checked In (and No-Show, with a hyphen). This caused multiple silent failures: a check-out guard that always blocked checkouts, an auto-checkout job that matched zero records, and — found via an ATF test in Phase 7 — a duplicate-check-in guard that never triggered. Resolved by querying the sys_choice table directly for ground truth rather than inferring values from sampled records, then standardizing every script reference against it.

3. GlideRecord.get() does not enforce ACLs in script context

An ATF regression test written to confirm a host cannot read another host's invitation initially failed, even though manual UI testing in Phase 3 showed the ACL working correctly. The cause: .get() succeeds regardless of the acting user's read access when called from a server-side script; canRead() is required to actually evaluate ACLs for the current session user. This is a meaningful platform nuance for anyone writing ACL-related automated tests.

Known Limitations

- REST API endpoints were built and code-reviewed but not fully verified via Postman due to Basic Auth configuration issues on the PDI — testing via OAuth would be a good next step
- Host lookup on the Service Portal check-in form uses a simple CONTAINS text match rather than a proper typeahead/reference picker
- The “Currently Checked In” Service Portal widget loads once per page view rather than auto-refreshing (would use polling or a websocket-based update in production)

Tech Stack

ServiceNow Platform · Flow Designer · Service Portal (Widgets / AngularJS) · Scripted REST API · Automated Test Framework (ATF) · Business Rules · Script Includes · Access Control (ACL) · Scheduled Jobs · Reporting / Dashboards

Repository structure

```
visitor-management-servicenow/
  README.md
  VMS_Project_Documentation.pdf
  update-set/
    VMS_Update_Set.xml
  screenshots/
    service-portal-checkin.png
    approval-flow.png
    acl-configuration.png
    atf-tests-passing.png
    dashboard.png
  diagrams/
    data-model-diagram.png
```

Setup / import instructions

1. Import update-set/VMS_Update_Set.xml into a ServiceNow instance via System Update Sets > Retrieved Update Sets > Import Update Set from XML
2. Preview and commit the update set
3. Assign the custom roles (front_desk, host, admin) to test users as needed
4. Configure the Service Portal URL suffix and verify Flow Designer flows are active